

Omokai Robotics Task

Architecture and Design Notes

How the pieces fit together

The system is four stages in a straight line: a CLI reads a natural language prompt, hands it to an LLM planner, the planner's output goes through a validator, and only a validated mission ever reaches the executor, which talks to PX4 over MAVSDK.

```
prompt -> llm_planner -> raw JSON -> validator -> Mission object -> executor -> PX4 SITL
```

Each stage lives in its own module (`src/llm_planner`, `src/validator`, `src/executor`), and none of them import each other. The only file that imports all three is `src/main.py`, which calls them in order. That's a deliberate choice, not an accident of how the code ended up organized it's the easiest way to prove, just by pointing at the imports, that the LLM has no path to the vehicle that skips the validator.

Why the LLM can't just fly the drone

The planner returns a plain Python dict. That dict means nothing to the executor the executor only accepts a `Mission` object, which is a frozen Pydantic model that can only be constructed by the validator after it passes every check. So the boundary isn't a convention the code is supposed to follow, it's a type the executor literally can't accept anything else in place of.

What the validator actually checks

Two things happen here. First, a normal Pydantic parse, which throws out wrong types, missing fields, and anything outside the allowed ranges for altitude, speed, or loop count. Second, a set of checks that run inside the model's own validator (so they can't be skipped by constructing the object a different way):

- mission has to start with a takeoff
- mission has to end with an RTL or a land no plan that just stops mid-air
- every waypoint has to sit inside the configured geofence
- altitude and speed are checked again against the mission's own constraints block, on top of the per-field limits
- loop repeat counts are capped
- a loop can't reference a waypoint that doesn't exist in the mission

If any of this fails, the validator returns a list of errors and stops. It does not try to fix the plan itself. Silently rewriting what the operator asked for defeats the point of having an audit trail, and it's the kind of "helpful" behavior that's much worse than just failing loudly.

The executor

Once a `Mission` object exists, the executor just walks its action list and issues the matching MAVSDK calls arm, take off to the given altitude, fly to each waypoint and wait until it's within about two meters, repeat a loop block the requested number of times, then RTL or land. There's no branching based on anything other than what's in the mission. Feed it the same JSON twice and you get the same flight both times, which is easy to prove with the `mission-file` flag: it replays a saved mission without touching the LLM at all.

One Gazebo, two build paths

Both the native setup and the Docker image run Gazebo Harmonic (`gz-sim 8.14.0`) against the same PX4 `gz_x500` target this isn't a two-simulator split, it's one simulator built two different ways for two different reasons.

Natively, PX4 was built directly on the host using `Tools/setup/ubuntu.sh`, which installs Harmonic through OSRF's own apt repository for whatever version is current at build time, then `make px4_sitl gz_x500` compiles PX4 against it. This is the fastest path to a working simulator on a machine you already control, but it isn't pinned to a specific commit or package version rerunning the same setup script six months from now could pull a newer PX4 or Gazebo release.

The Docker image exists specifically to fix that. `docker/Dockerfile.px4` clones PX4 at a single pinned commit (`e0137fe7a7a1be179db7d04b8703ef77c71a656e`) and installs an exact `gz-harmonic` package version, so the container gives the same build regardless of when or where it's run. The compose file's healthcheck polls Gazebo's own `/clock` topic (`gz topic -l | grep /clock`) to confirm the simulator is actually up, rather than sleeping a fixed number of seconds and assuming it's ready.

So the difference between the two isn't the simulator it's reproducibility. The native build is quick to stand up but not pinned; the Docker build is slower to build the first time but gives the examiner the exact same environment every time.

Challenges not implemented how I'd approach them

Multi-agent formations

The mission schema would need a formation block: a list of vehicle IDs and their offset from a leader (north/east offsets are enough for a flat formation). The validator would check that every vehicle's absolute position home plus offset still sits inside the geofence, not just the leader's path.

On the simulation side, PX4 SITL supports running several instances at once, each on its own MAVLink port, so each vehicle gets its own MAVSDK connection and its own asyncio task in the executor. To keep the formation from stringing out, I'd put a barrier between waypoints every vehicle has to report arrival before any of them moves to the next point. Followers would run their own validator pass on their offset-adjusted path before arming, so a formation that would push one drone outside the geofence gets rejected before anything takes off, not mid-flight.

SLAM / autonomous navigation

This one is really a planning problem sitting in front of the same validator. A depth camera on the model feeds a VIO/SLAM stack (something like ORB-SLAM3 or OpenVINS) to build a local map, and a planner RRT* or A* turns that map into a sequence of waypoints around whatever obstacles show up. Those waypoints still go through the same `validator.validate()` call as anything else; an autonomously planned path isn't exempt from the geofence or altitude limits just because a human didn't type it. If the map changes mid-mission, the planner re-runs and the new path goes through validation again before the executor is allowed to switch to it. For actual flight control at that point, waypoint-mission mode isn't fine-grained enough I'd move to MAVSDK's offboard velocity setpoints at roughly 10 Hz.

Vision target detection and follow

I didn't get to building this one `src/` only has the planner, validator, executor, and schemas, no vision code. The design I'd follow:

A vision node subscribes to the simulated camera feed and runs YOLOv8 on each frame. On a detection of the configured target class, it saves a snapshot and estimates the target's ground position from its pixel location plus the drone's own telemetry. That estimate becomes a single `GotoWaypointAction`, which same as everywhere else in this project would still have to go through the validator before the executor acts on it. The vision system wouldn't be a special case; it'd just be another source of proposed waypoints, sitting in front of the same guardrail the LLM planner sits behind.

Scaling this to something real

The honest gaps between this demo and a real deployment: real vehicles need a real geofence tied to actual no-fly data, not a hardcoded bounding box; the retry loop that patches over a local LLM's occasional bad JSON needs a hard ceiling and a fallback to a human operator, not just more attempts; and running multiple vehicles at once needs a real fleet manager rather than one `asyncio` task per drone in a single process. Logging every command is a good start on auditability, but a real system needs those logs shipped somewhere durable, not sitting in a local file that disappears with the container.